DSA0309 – NATURAL LANGUAGE PROCESSING

ASSIGNMENT – 2 (CO2)

Build and Evaluate an N-gram Language Model and a Part-of-Speech Tagger for a Public Transport Enquiry Helpline

Name: Yoga Madha A

Register Number:192425058

Department / Section:B.Tech AI&ML(3rd Year)

GitHub Link: https://github.com/YOGAMADHA/DSA0309-NLP/tree/main/Assignment%202

Dataset, assumptions and reproducibility

This report follows the 12 tasks and terminology in the supplied DSA03 Assignment-2 question paper. The accompanying Python program creates a deterministic synthetic public-transport enquiry corpus using seed 42. It generates 20,000 utterances and splits them by utterance into 16,000 training, 2,000 development and 2,000 test utterances. This prevents an utterance from appearing in more than one split. The measured output from the included run is shown below.

Corpus measure	Measured value
Total utterances	20,000
Training / development / test	16,000 / 2,000 / 2,000
Training word tokens	153,601
Training word types	47
Development OOV rate	0.0000
Observed bigram / trigram types	298 / 1,576
Add-one / add-k (0.1) bigram perplexity	4.173 / 4.130

Important limitation: the paper allows real or synthetic corpus data, but it additionally requires at least 3,000 hand-tagged tokens, 500 tokens independently double-annotated by two people, and 200 recogniser sets with five hypotheses and a reference transcript. The generated corpus has automatic labels and no real ASR alternatives. Therefore human agreement, Cohen's kappa, WER rescoring, and claims based on independent annotation are explicitly identified as data still to be collected. They are not invented.

TASK 1 – Counting Words and Building the Vocabulary

1.1 Type, token, lemma and word form

A token is one occurrence of a word in the corpus. For example, in “bus bus ticket”, there are three tokens. A type is a distinct vocabulary item, so the same example has two types: bus and ticket. A word form is the exact surface form found in the transcript, such as “books” or “booked”. A lemma is the base dictionary form; “book”, “books” and “booking” may share the lemma BOOK depending on the lemmatiser.

1.2 Counting and tokenisation decisions

The program converts generated utterances to lower case and splits on whitespace. Thus “Vellore” and “vellore” are treated as the same form. Punctuation is not generated in the synthetic corpus. A time such as “6:30” is retained as one token rather than split into “6”, “:” and “30”. Disfluencies such as “uh”, repetitions and false starts are absent from the generated data. Start and end markers <s> and </s> are added for n-gram modelling but are not counted as lexical word tokens.

1.3 Counts from the included corpus

The run contains 153,601 training tokens and 47 training types. Since the synthetic generator does not create inflected variants, its estimated lemma count and word-form count are both 47. This equality is a property of this simple synthetic vocabulary, not a general property of English.

1.4 Effect of alternate tokenisation

Decision A (used): lowercase and whitespace tokenisation; 6:30 is one token. Decision B (alternative): preserve case and split time expressions into numeric and punctuation components. Decision B can increase the number of distinct forms because “Vellore” and “vellore” become separate types and “6”, “:” and “30” are separate token forms. For this corpus, case-folding is the main relevant decision because the generator lowercases words before counting; time-splitting has no effect on the current token set unless time strings are split deliberately.

1.5 Zipf’s law

Zipf’s law states approximately that the frequency of a word is inversely proportional to its rank: $f(r) \approx C/r^s$, where r is the rank, C is a constant and s is often near 1. A log-frequency versus log-rank plot should therefore be approximately a straight descending line. The measured log-log slope for this generated corpus is -0.571. It is not close to the idealised −1 because the corpus is generated from a small set of fixed templates and a restricted vocabulary. Repeated template words and the limited number of contexts create a distribution different from naturally occurring speech. The ten most frequent words are: please (17923), today (17881), is (6404), bus (4309), metro (4266), train (4195), the (4054), your (4012), a (3998), my (3936).

1.6 Unknown-word thresholds and OOV

The threshold rule retains a word in the vocabulary if its training count is at least the selected threshold. Words below the threshold are mapped to <UNK>. The output JSON includes vocabulary size and the proportion of training tokens mapped to <UNK> for thresholds 1, 2, 3 and 5. The development OOV rate is calculated as the number of development tokens absent from the training vocabulary divided by all development tokens. The measured rate is 0.0000. It is zero because the synthetic generator draws from the same fixed vocabulary in every split. A real expanding transport service would have unseen station and train names, so the OOV rate would be higher.

Why it matters: an unseen destination may be the most important word in a caller’s request. If it is mapped to a generic <UNK>, the model may lose the exact origin or destination even if it recognises the surrounding words.

TASK 2 – Unsmoothed N-gram Models

2.1 Chain rule and Markov assumptions

For a sequence $W=w1,w2,\ldots,wn$, the chain rule is $P(W)=P(w1)P(w2|w1)\ldots P(wn|w1,\ldots,w(n-1))$. The chain rule is exact. The unigram approximation replaces each conditional probability with

P(wi), assuming words are independent. The bigram approximation uses P(wi|w(i−1)), assuming the next word depends only on the immediately preceding word. The trigram approximation uses P(wi|w(i−2),w(i−1)), assuming the next word depends only on the previous two words. Each higher order preserves more local context but requires more counts.

2.2 Maximum-likelihood estimates

Unigram: P(w)=C(w)/N. Bigram: P(wi|w(i−1))=C(w(i−1),wi)/C(w(i−1)). Trigram: P(wi|w(i−2),w(i−1))=C(w(i−2),w(i−1),wi)/C(w(i−2),w(i−1)). Here C(·) is the count in training data. The program adds <s> and </s> around each utterance and builds unigram, bigram and trigram frequency tables.

2.3 Worked probability calculation

The program calculates the probability of three example transport enquiries and stores every bigram factor in outputs/results.json. For a sentence “please book the bus ticket to vellore”, the bigram probability is the product P(please|<s>) × P(book|please) × P(the|book) × P(bus|the) × P(ticket|bus) × P(to|ticket) × P(vellore|to) × P(</s>|vellore). Each factor is its bigram count divided by the count of its left context. If any factor is zero, the entire product is zero.

2.4 Why zero probability is a problem

A perfectly understandable sentence can receive zero probability when one adjacent word pair did not occur in training. For example, if “ticket Vellore” was not observed as a bigram, P(Vellore|ticket)=0 under the unsmoothed model. This does not prove that the phrase is impossible; it only shows that the training sample did not contain that pair.

2.5 Sparsity

The run observed 298 distinct bigram types and 1,576 distinct trigram types. The development unseen-bigram rate is 0.0000. The possible number of bigrams grows roughly with V^2 and possible trigrams with V^3, while only a small portion can occur in a finite corpus. This is why higher-order models are sparse.

2.6 Generation

A unigram generator samples words independently and normally produces an incoherent sequence. A bigram generator uses the last generated word to choose the next word and therefore preserves local pairs. A trigram generator uses the previous two words and can produce more locally plausible sequences. The cost is increased model size and more unseen contexts. The provided program builds the counts needed for these generators; its main output focuses on model evaluation rather than presenting randomly generated sentences as meaningful helpline calls.

TASK 3 – Smoothing and Discounting

3.1 Why smoothing is needed

An unsmoothed model assigns zero to an unseen event. In a finite corpus, absence from the sample is not proof that a word sequence cannot occur. Smoothing reserves some probability mass for unseen events. For each context, all conditional probabilities must be non-negative and sum to one.

3.2 Add-one and add-k

Add-one (Laplace) smoothing: $P(w|h)=(C(h,w)+1)/(C(h)+V)$. Add-k smoothing: $P(w|h)=(C(h,w)+k)/(C(h)+kV)$, where V is vocabulary size and $k>0$. Add-one is the special case $k=1$. Smaller k discounts observed events less severely and is often more suitable when the vocabulary is large.

3.3 Discount calculation

Suppose a context occurs $C(h)=20$ times and the vocabulary has $V=50$ possible next words. Add-one uses denominator $20+50=70$. If an observed continuation has count 8, its probability becomes 9/70 rather than 8/20. The seen event's probability is reduced to make room for the pseudo-counts assigned to all other vocabulary items. This example illustrates why add-one can over-discount when V is large; the values here are a worked illustration, not a corpus measurement.

3.4 Good–Turing discounting

Let N_r be the number of distinct n-gram types that occur exactly r times. Good–Turing replaces an observed count r with adjusted count $r^*=(r+1)N_{(r+1)}/N_r$. The total probability reserved for unseen n-grams is approximately N_1/N, where N_1 is the number of singleton n-gram types and N is the total number of observed n-gram tokens. A frequency-of-frequencies table is needed to calculate the discount. In practice, sparse high-frequency bins are smoothed or a cutoff is used because $N_{(r+1)}$ may be zero or unreliable.

3.5 Comparison and selection

The included run measured development bigram perplexity of 4.173 for add-one and 4.130 for add-k with $k=0.1$. Lower perplexity is better on the same held-out set and vocabulary. In this run add-k is slightly lower. This supports add-k as the better of these two tested settings for this synthetic split. A full Good–Turing score and a comparison on genuine transport transcripts remain to be run; they cannot be inferred from the two measurements above.

TASK 4 – Backoff and Deleted Interpolation

4.1 Backoff versus interpolation

Backoff consults a lower-order model only when the higher-order context is unavailable. Interpolation combines multiple orders for every prediction. For example, a trigram model may

use a trigram estimate when reliable and otherwise back off to a bigram or unigram. Linear interpolation uses $P(w|u,v)=\lambda 3P(w|u,v)+\lambda 2P(w|v)+\lambda 1P(w)$, with non-negative weights whose sum is one.

4.2 Katz backoff

Katz backoff discounts the probability of observed n-grams and distributes the remaining probability mass among unseen continuations using a backoff distribution. The $\alpha(h)$ factor normalises the lower-order probabilities over the unseen continuations for context h. Without α, the total probability for the context may not equal one. Good–Turing discounting is commonly used to estimate the discount.

4.3 Stupid backoff

Stupid backoff uses the higher-order count ratio when available and otherwise multiplies the lower-order score by a fixed factor, often 0.4. It is computationally simple and useful for ranking candidates. It is not a probability model because its scores are not normalised to sum to one.

4.4 Deleted interpolation

Deleted interpolation estimates λ weights using held-out events. For each held-out n-gram, temporarily remove its contribution from the training counts, compare the unigram, bigram and trigram estimates, and assign credit to the order that predicts it best. Aggregate credits and normalise them to obtain $\lambda 1$, $\lambda 2$ and $\lambda 3$. Context-count bands can be used because high-count contexts support higher-order estimates, while low-count contexts need more lower-order weight.

4.5 Worked example

Assume the held-out procedure estimates $\lambda 1=0.15$, $\lambda 2=0.35$ and $\lambda 3=0.50$ for a well-observed context. If the trigram probability is 0.20, bigram probability is 0.10 and unigram probability is 0.02, the interpolated value is $(0.50\times 0.20)+(0.35\times 0.10)+(0.15\times 0.02)=0.138$. These weights and probabilities are illustrative arithmetic only; the final assignment should replace them with values estimated from the corpus.

4.6 Comparison

Backoff can be efficient when longer contexts are frequently observed, while interpolation is robust because lower-order evidence always contributes. A proper comparison must calculate perplexity on the same held-out tokens. The current program does not claim measured Katz or deleted-interpolation weights; these require the additional count-discount and held-out optimisation procedures described above.

TASK 5 – Entropy, Cross-Entropy and Perplexity

5.1 Definitions

Entropy $H(P)=-\Sigma_x P(x)\log_2 P(x)$ measures uncertainty in the true distribution. Cross-entropy $H(P,Q)=-\Sigma_x P(x)\log_2 Q(x)$ measures the average coding cost when model Q predicts data generated by P. Perplexity is 2^H when H is measured in bits, or $\exp(H)$ when natural logarithms are used. Lower cross-entropy and perplexity indicate better predictive performance on the same data.

5.2 Meaning of perplexity 90

A perplexity of 90 means the model's average uncertainty is equivalent to choosing among 90 equally likely alternatives at each prediction step. It does not mean that exactly 90 words are possible or that the model is 90% accurate.

5.3 Hand calculations

For a uniform distribution over four outcomes, each probability is 0.25 and entropy is $-4(0.25 \log_2 0.25)=2$ bits. For a sharply peaked distribution (0.97, 0.01, 0.01, 0.01), entropy is approximately 0.242 bits. The uniform distribution is difficult to predict; the peaked distribution is easier because one outcome dominates. These are explanatory distributions, not measurements from the corpus.

5.4 Measured perplexity

The included run measured add-one bigram perplexity 4.173 and add-k (0.1) bigram perplexity 4.130 on the same development split. Per-word cross-entropy in natural-log units is ln(perplexity); per-utterance loss should be reported separately by summing log probabilities within each utterance and then averaging over utterances.

5.5 Fair comparisons and ASR rescoring

Models can be compared fairly only when they use the same tokenisation, vocabulary, <UNK> policy, sentence boundaries and held-out text. An unfair comparison occurs if one model maps many rare words to <UNK> but another retains them as separate vocabulary items: the prediction problem is no longer the same. The requested WER experiment needs 200 recogniser utterances, five hypotheses per utterance and a reference transcript. $WER=(S+D+I)/N$, where S is substitutions, D deletions, I insertions and N reference words. Since these data are not included, no before/after WER is fabricated.

TASK 6 – English Word Classes and Tagset Design

6.1 Open and closed classes

Open classes accept new words readily: nouns, lexical verbs, adjectives and adverbs. Closed classes contain a relatively fixed set of grammatical words: determiners, pronouns, prepositions,

conjunctions, auxiliaries, modals and particles. Closed-class items are frequent and carry grammatical relationships, so incorrect tagging of them can affect many sentences.

6.2 Morphological and distributional tests

Nouns can often follow determiners and take plural marking: “the ticket”, “tickets”. Verbs can change tense or agree with a subject: “book”, “booked”, “books”. Adjectives can occur before nouns and may take degree marking: “available service”, “more available”. Adverbs modify verbs or adjectives: “travel quickly”, “very late”. These tests must be combined with context because a word can belong to more than one class.

6.3 Ambiguous transport words

Book: noun in “the book” but verb in “book a ticket”. Return: noun in “a return ticket” but verb in “return tomorrow”. Stand: noun in “bus stand” but verb in “please stand”. Train: noun in “the train” but verb in “train the staff”. Time: noun in “the time” but verb in “time the journey”. Context and syntactic position help resolve the class.

6.4 Proposed domain tagset (24 tags)

Tag	Meaning	Penn Treebank mapping
NN	Common singular noun	NN
NNS	Plural noun	NNS
NNP	Station/place proper noun	NNP
NNPS	Plural proper noun	NNPS
VB	Base verb	VB
VBD	Past-tense verb	VBD
VBG	Gerund/present participle	VBG
VBN	Past participle	VBN
VBP	Non-third-person present verb	VBP
VBZ	Third-person singular present verb	VBZ
JJ	Adjective	JJ
JJR	Comparative adjective	JJR
JJS	Superlative adjective	JJS
RB	Adverb	RB
RBR	Comparative adverb	RBR
RBS	Superlative adverb	RBS
DT	Determiner	DT
PRP	Personal pronoun	PRP
IN	Preposition/subordinating conjunction	IN
CC	Coordinating conjunction	CC
CD	Number/time expression	CD
MD	Modal auxiliary	MD
UH	Interjection	UH
FW	Foreign/code-mixed token	FW

6.5 Distinctions retained and not retained

The tagset separates proper station names from common nouns because station names are important to origin/destination extraction. It keeps verb inflection distinctions because tense and agreement can help parsing. It includes FW for code-mixed or foreign tokens. It does not create separate tags for every station-name subtype (bus stop, metro station, railway station), every pronoun gender, or every punctuation mark because those distinctions are not required for the core POS task. A separate named-entity layer can capture station type without expanding POS tags.

6.6 Annotation and agreement

The assignment requires at least 3,000 tokens to be hand-tagged and at least 500 tokens independently tagged by two annotators. Raw agreement = number of matching tags / total double-tagged tokens × 100. Cohen's kappa is $\kappa=(Po-Pe)/(1-Pe)$, where Po is observed agreement and Pe is expected agreement by chance from the annotators' tag distributions. The synthetic script uses deterministic labels and therefore cannot be used to claim independent human agreement or a valid human κ.

TASK 7 – Rule-Based Part-of-Speech Tagging

7.1 Two-stage architecture

A rule-based tagger first uses a lexicon/dictionary to assign one or more possible tags to every word. The second stage applies contextual constraints to remove tags that do not fit the surrounding words. Unknown words may receive a default noun/proper-noun tag or a suffix-based candidate.

7.2 Fifteen explicit example rules

No.	Condition inspected	Action
1	Previous tag is DT	Prefer NN or JJ; remove VB where context disallows it
2	Previous tag is PRP and current word is a lexical verb	Choose VB/VBP
3	Previous tag is TO	Choose VB
4	Previous tag is IN	Prefer NN/NNP after the preposition
5	Current word matches a known station gazetteer entry	Choose NNP
6	Current word matches a time or numeric pattern	Choose CD
7	Current word is a known determiner	Choose DT
8	Current word is a known preposition	Choose IN
9	Current word is a known	Choose PRP

	personal pronoun	
10	Current word ends in -ly	Prefer RB
11	Current word ends in -ed	Prefer VBD/VBN according to context
12	Current word ends in -ing and follows an auxiliary	Prefer VBG
13	Current word follows a modal	Choose VB
14	Next word is a noun and current word is an adjective candidate	Choose JJ
15	Previous tag is DT and current word is “book”	Choose NN; override the verb default

7.3 Dictionary ambiguity

A word such as “book” should have both NN and VB candidates in the dictionary. A context rule then chooses between them. The program’s domain lexicon can be extended with new station names and service terms.

7.4 Conflict and rule order

Two rules may conflict: a lexical default may classify “book” as VB, while the preceding determiner “the” supports NN. The contextual determiner rule should have higher priority in “the book”; a verb-context rule should win in “book a ticket”. Rule order matters and can become difficult to maintain as exceptions accumulate.

7.5 Evaluation

Overall accuracy = correctly tagged tokens / all evaluated tokens. Ambiguous-token accuracy uses only tokens that have more than one possible tag. It is especially informative because unambiguous words are easy and can inflate overall accuracy. The five most common error classes should be extracted from a real test confusion matrix; the synthetic heuristic labels do not provide independent gold labels for a meaningful error analysis.

TASK 8 – Stochastic (HMM) Tagging and the Viterbi Algorithm

8.1 HMM equation

By Bayes’ rule, the best tag sequence is argmax_T P(T|W)=argmax_T P(W|T)P(T), since P(W) is constant for all candidate tag sequences. With the HMM assumptions, this becomes argmax_(t1…tn) Πi P(wi|ti)P(ti|t(i−1)). The transition probability models tag-to-tag sequence; the emission probability models a word given its tag.

8.2 Estimating probabilities

Transition estimate: P(ti|t(i−1))=C(t(i−1),ti)/C(t(i−1)). Emission estimate: P(w|t)=C(t,w)/C(t). Additive smoothing can be applied to avoid zero transition or emission probabilities. For each

previous tag, the transition probabilities over all next tags, including sentence end, must sum to one.

8.3 Viterbi recurrence

Initialise $\delta 1(t)=P(t|<s>)P(w1|t)$. For later positions, $\delta i(t)=\max$ over previous tags u of $[\delta(i-1)(u)P(t|u)P(wi|t)]$. Store the best previous tag as a back-pointer. At the final position, select the best ending tag and follow back-pointers in reverse order. Log probabilities are used in code to avoid numerical underflow.

8.4 Example run

The included script runs Viterbi on "please book the bus ticket to vellore" and prints the resulting word/tag sequence in the console output and results.json. Its tag dictionary is rule-generated from the synthetic vocabulary, so the output demonstrates the algorithm rather than a human-validated tagging result.

8.5 Complexity and baselines

A most-frequent-tag baseline assigns the most common tag to each word. Exhaustive enumeration considers every possible tag sequence and has exponential growth O(T^n). Viterbi uses dynamic programming with approximately $O(nT^2)$ time and O(nT) trellis storage. Unknown words can be handled with suffix patterns, capitalisation, a character model, gazetteers and open-class priors. The OOV-only accuracy must be measured on an independently labelled test set.

TASK 9 – Transformation-Based (Brill) Tagging

9.1 Learning process

Brill tagging starts with an initial tag sequence, commonly from a most-frequent-tag baseline. It generates candidate transformation rules from templates. Each candidate is scored by how many current errors it corrects minus how many errors it introduces. The best positive-gain rule is applied, and the process repeats until the gain falls below a threshold or a maximum number of rules is reached.

9.2 Six example templates

Template	Example transformation
Previous tag	Change NN to VB when previous tag is PRP
Following tag	Change NN to JJ when next tag is NN
Previous word	Change NN to VB when previous word is "to"
Following word	Change NN to VB when next word is "a" and context supports a verb
Word suffix	Change NN to VBG when word ends in "-ing"
Capitalisation / word form	Change NN to NNP when token is capitalised and appears in the station gazetteer

9.3 Readability and overfitting

Learned rules are readable because they express explicit linguistic conditions. HMMs instead store many numerical transition and emission values. A general rule such as “change NN to VB after PRP” may reflect a useful grammatical pattern. A rule that corrects only one rare spelling in the training set may be an artefact. Plot accuracy against the number of applied transformations; if training accuracy continues to rise while development accuracy falls, later rules are overfitting.

9.4 Required measurements

The first ten learned transformations, corrected/introduced error counts and the overfitting point must come from running a Brill learner on annotated training and development data. They are not claimed from the current automatic synthetic labels because that would produce a circular evaluation.

TASK 10 – Implement and Test the Models

10.1 Files and execution

The ZIP contains code/run_assignment.py, README.txt, outputs/results.json, outputs/results.txt, outputs/run_console.txt and outputs/output_screenshot.png. It uses Python’s standard library and a fixed random seed. From the extracted package root, run: python code/run_assignment.py.

10.2 Implemented in the supplied prototype

The program generates the synthetic corpus; splits it 80/10/10; counts tokens and vocabulary; computes Zipf slope and OOV rate; builds unigram, bigram and trigram counts; calculates unsmoothed bigram sentence probabilities; evaluates add-one and add-k bigram perplexity; estimates simple HMM transition/emission counts; and runs Viterbi.

10.3 Further implementation needed for complete rubric coverage

The question paper asks for every smoothing method, Katz backoff, stupid backoff, linear/deleted interpolation, rule-based tagging, HMM tagging and Brill tagging, as well as runtime, memory, tagging speed, unit tests and two plots. The included prototype is a starting implementation and does not yet provide all of those advanced experiments. In particular, human annotation and the 200 ASR hypothesis sets cannot be generated by code alone. Do not describe these items as completed unless they are implemented and measured.

TASK 11 – Compare, Recommend and Defend the Design

11.1 Language-model recommendation

From the two measured smoothing settings in this prototype, add-k with k=0.1 has a slightly lower development perplexity than add-one. Therefore add-k is the preferred baseline for this synthetic experiment. Before deployment, it should be compared against Good–Turing, Katz

backoff and deleted interpolation using the same real held-out transport transcripts and vocabulary.

11.2 Tagger recommendation

An HMM with Viterbi is a reasonable baseline because it chooses a globally consistent tag sequence rather than making each decision independently. A practical system can combine it with a small rule layer and station gazetteer for high-value domain terms. The final choice must be supported by independent tagging accuracy, ambiguous-token accuracy, OOV accuracy, speed and maintainability.

11.3 Operational failure and human effort

If the system fails to identify a destination, it should ask the caller to repeat or confirm the station rather than silently extracting a wrong place. Manual work includes transcript cleaning, tagset design, double annotation, disagreement resolution, rule writing and error review. Once common rule patterns are covered, adding independently annotated examples is often more useful than adding many one-off rules.

TASK 12 – Day-4 Design Challenge – Re-evaluate and Adapt the Design

A. Impact Analysis

The expanded service adds metro and intercity rail, new station/train names and code-mixed utterances such as “Chennai to Madurai bus timing enna”. The assumptions of a stable closed vocabulary and English-only word classes no longer hold. More than 15% OOV means many new tokens have no learned word-level statistics. These are particularly harmful when they are station names because they carry the caller’s destination or origin.

Perplexity is expected to increase because the model must predict unseen words. Tagging accuracy is expected to decrease because an unknown station name may be labelled as a generic noun rather than a proper noun. The precise degradation must be measured on a new labelled rail/code-mixed test set; it cannot be calculated honestly from the original fixed-vocabulary test split alone.

B. Evaluation of the Original Design

Example 1: “metro fare to Koyambedu” contains a station name that may be absent from the original vocabulary. Example 2: “Chennai to Madurai bus timing enna” includes the code-mixed token “enna”. Example 3: “train ticket from Tambaram tomorrow” contains a potentially unseen station name. A closed-vocabulary model may map the names to <UNK>; an English-only tagger may fail to assign the correct proper-noun or foreign-token tag. The first failure is often vocabulary recognition, followed by incorrect tagging and entity extraction.

C. Adaptation or Redesign

Decision: MODIFY the original design. Keep the n-gram and HMM foundations, but add: (1) a station/train gazetteer; (2) spelling normalisation for ASR variants; (3) a code-mixed FW tag; (4) character n-gram or subword modelling for unseen names; (5) updated mixed-domain training data; (6) re-estimation of smoothing and interpolation weights; and (7) monitoring for OOV and perplexity changes. Update the gazetteer monthly and retrain when new data or monitored performance indicates drift.

D. Evidence-Based Defence

This modification is more economical than replacing every component. The existing models still capture common local English patterns, while gazetteers and character features address proper nouns and unseen spellings. Evaluate the modified design against the original using identical new test calls and report OOV rate, perplexity, WER after rescoring, proper-noun tagging accuracy, code-mixed tagging accuracy, memory and latency. Remaining limitations include long-distance dependencies, semantic ambiguity and the fact that a finite-order n-gram only conditions on a short history.

Final measured results from the included run

Measure	Value
Utterances	20000
Training tokens	153601
Training types	47
Development OOV rate	0.0
Zipf log-log slope	-0.571
Bigram types	298
Trigram types	1576
Unseen development bigram rate	0.0
Add-one bigram perplexity	4.173
Add-k 0.1 bigram perplexity	4.13

These are reproducible synthetic-corpus measurements from the supplied code. Do not present them as measurements from actual transport helpline recordings.